

RESEARCH ARTICLE | APRIL 01 2026

Modeling memristor-based neural networks with Manhattan update: Trade-offs in learning performance and energy consumption

Walter Quiñonez ; María José Sánchez ; Diego Rubi  



APL Electronic Devices 2, 026102 (2026)

<https://doi.org/10.1063/5.0310714>



Articles You May Be Interested In

An electronic system for simulation of neural networks with a micro-second real time constraint

AIP Conf. Proc. (August 2001)

Machine learning-based prediction of species mass fraction and flame characteristics in partially premixed turbulent jet flame

Physics of Fluids (July 2025)

HRV stress analysis using k-NN clustering technique

AIP Conf. Proc. (October 2023)

Modeling memristor-based neural networks with Manhattan update: Trade-offs in learning performance and energy consumption

Cite as: APL Electron. Devices 2, 026102 (2026); doi: 10.1063/5.0310714

Submitted: 4 November 2025 • Accepted: 16 March 2026 •

Published Online: 1 April 2026



Walter Quiñonez,^{1,2} María José Sánchez,^{3,4} and Diego Rubi^{1,a)}

AFFILIATIONS

¹Laboratorio de Ablación Láser (INN-CONICET-CNEA), Gral. Paz 1499 (1650), Buenos Aires, Argentina

²Universidad de Buenos Aires, Facultad de Ciencias Exactas y Naturales, Departamento de Física, Ciudad Autónoma de Buenos Aires, Buenos Aires C1428, Argentina

³Centro Atómico Bariloche and Instituto Balseiro (UNCuyo), San Carlos de Bariloche, Río Negro 8400, Argentina

⁴Instituto de Nanociencia y Nanotecnología (INN-CONICET-CNEA), Nodo Bariloche, Av. Bustillo s/n (8400), San Carlos de Bariloche, Argentina

^{a)}Author to whom correspondence should be addressed: diego.rubi@gmail.com

ABSTRACT

We present a systematic study of memristor-based neural networks trained with the hardware-friendly Manhattan update rule, focusing on the trade-offs between learning performance and energy consumption. Using realistic models of potentiation/depression (P/D) curves, we evaluate the impact of nonlinearity (NLI), conductance range, and number of accessible levels on both a single perceptron and a multi-layer perceptron (MLP) trained on the MNIST dataset. Our results show that SPs tolerate P/D nonlinearity up to $NLI \leq 10^{-2}$, while MLPs require stricter conditions of $NLI \leq 10^{-3}$ to preserve accuracy. Increasing the number of discrete conductance states improves convergence, effectively acting as a finer learning rate. We further propose a strategy where one memristor of each differential pair is fixed, reducing redundant memristor conductance updates. This approach lowers training energy by nearly 50% in MLP with little to no loss in accuracy. Our findings highlight the importance of device–algorithm co-design in enabling scalable, low-power neuromorphic hardware for edge artificial intelligence (AI) applications.

© 2026 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>). <https://doi.org/10.1063/5.0310714>

I. INTRODUCTION

Artificial neural networks (ANNs) have become indispensable tools in modern computing,¹ enabling breakthroughs in fields such as image recognition, autonomous systems, speech processing, and personalized recommendations.^{2–5} These networks rely on a large number of interconnected computational nodes and adjustable weights to learn complex patterns from data. However, the hardware implementation of such networks remains constrained by the limitations of traditional von Neumann architectures, where the separation between memory and computation leads to significant inefficiencies due to frequent data transfers.^{6,7}

To address this “memory wall,” alternative computing paradigms are being actively explored. Among them, memristor-based hardware offers a promising route by combining computation

and memory storage within a single nanoscale device.^{6,8–10} Memristors are resistive switching devices whose conductance can be modulated by electrical pulses and retained without power,^{11–16} allowing them to act as synaptic weights in hardware neural networks.^{17–23} When arranged in crossbar arrays, memristors naturally enable in-memory vector-matrix multiplications (VMM)^{24–26}—the computational bottleneck in most ANN operations—while drastically reducing energy consumption and latency. However, the performance of ANNs is usually affected by the so-called *non-idealities* of the memristive devices that constitute their building blocks. These non-idealities include non-linear potentiation–depression curves, a discrete number of accessible conductance levels within a limited range defined by the G_{MAX} and G_{MIN} values, in addition to device-to-device and cycle-to-cycle variations.^{19,27–36} While memristive systems have shown considerable promise in accelerating

inference tasks,^{17,37} enabling online training remains a more demanding challenge.^{17,18,20} Training directly on hardware requires robust and reliable weight update mechanisms that cope with the analog nature and physical non-idealities of memristor devices. In this work, we simulate the behavior of memristor-based single and multi-layer perceptrons with *in situ* training, focusing specifically on how device-level characteristics affect network performance in terms of convergence behavior, classification accuracy, and energy efficiency.

The Manhattan update rule^{18,38} appears as a simplified gradient-based approach particularly suited for hardware applications. This rule updates weights incrementally in fixed steps based on the sign of the gradient, making it well-suited for memristive devices that support discrete conductance levels and can be programmed with single pulses. Compared to more complex update schemes, the Manhattan rule reduces circuit overhead and increases robustness against device variability, while still maintaining learning performance in proof-of-concept simple networks, such as perceptrons.^{17,18}

Despite its simplicity of implementation and inherent advantages, no systematic studies on the suitability of more complex ANNs using the Manhattan rule under realistic constraints for memristors have been reported in the literature. This paper aims to fill this gap by numerically investigating the influence of three key physical parameters on ANN performance under the Manhattan rule:

- (i) **Linearity of potentiation and depression curves.** Ideal weight updates assume that memristor conductance changes are proportional to the number of programming pulses. In real memristors, however, the update curves are often highly nonlinear, which can distort learning dynamics and impede convergence;
- (ii) **Limited conductivity windows (from G_{MIN} to G_{MAX}).** The ratio between the maximum and minimum conductance values defines the dynamic range available for weight encoding. A narrow window can limit learning capacity or lead to premature saturation, while a wider window allows finer control but may increase sensitivity to noise;
- (iii) **Multilevel conductance resolution.** The number of stable intermediate states in the memristor's conductance directly affects the granularity—defined as the minimum resolvable change in conductance that the device can reliably achieve—of weight updates.³⁹ Higher resolution can improve accuracy but also demands tighter control over device programming.

By incorporating realistic device models into our simulations, we capture the impact of these non-idealities on ANN performance. We benchmark both single and multi-layer perceptrons using the MNIST handwritten digits database under various parameter regimes to evaluate the trade-offs involved in *in situ* learning. In addition, we analyze the energy consumption associated with training, accounting for both read and write operations, to assess the practical viability of these systems for low-power edge computing applications.

This work provides a comprehensive view of the challenges and opportunities in implementing memristive neural networks with online training. Our study contributes to the co-design of efficient, adaptive neuromorphic hardware. The use of the Manhattan update

rule offers a realistic and hardware-friendly pathway toward scalable and trainable memristive AI systems.

II. METHODS

Artificial neural networks (ANNs) consist of interconnected layers of neurons, where the strength or synaptic weight (w_{ij}) of the connection between neurons is iteratively trained for a specific task. A standard architecture for ANNs in machine learning is the feed-forward architecture, in which neurons in one layer connect only to neurons in the subsequent layer. This means that there are no connections between neurons within the same layer or with previous layers and no recurrent loops.⁴⁰ When an ANN consists of only one layer beyond the input layer, it is referred to as a single-layer perceptron (SP). Networks, including one or more hidden layers, are typically classified as multilayer perceptrons (MLP) and may be considered deep neural networks when the depth becomes sufficiently large. Following a basic training scheme, a training dataset is fed into the network, and its performance is evaluated using an activation function. The synaptic weights are then adjusted by minimizing a loss function.

We aim to simulate the online training of a MLP physically implemented on memristor crossbar arrays, where the conductances of memristors are linked to the MLP synaptic weights. Figure 1(a) displays a sketch of an ANN, including a single hidden layer. The weights between the input and hidden layers are represented by the matrix elements $w1_{ij}$, while the weights between the hidden and output layers are represented by $w2_{ij}$.

In Fig. 1(b), we present the electrical scheme of a crossbar array, which serves as the hardware implementation of the MLP shown in Fig. 1(a). Following the approach proposed by Prezioso *et al.*,¹⁸ the synaptic weights of the ANN are encoded as the conductances of differential memristor pairs,¹⁷ such that $wl_{ij} = Gl_{ij}^+ - Gl_{ij}^-$, with $l = 1, 2$. This scheme enables both positive and negative weights, thereby overcoming the intrinsic limitation that individual device conductances are strictly positive. The trade-off is a doubling of the required memristors, with the subtraction of currents physically realized by a differential amplifier. Following this scheme, the hidden layer of the MLP is represented by the conductance matrices $G1_{ij}^+$ and $G1_{ij}^-$, while the output layer is represented by $G2_{ij}^+$ and $G2_{ij}^-$. The output current I_i^1 of the hidden layer can be calculated using Ohm's and Kirchhoff's laws as

$$I_i^1 = \sum_{j=1}^m (G1_{ij}^+ - G1_{ij}^-) V_j, \quad (1)$$

where V_j is the input data vector encoded as voltage values, and we consider that the layer contains $m \times 2k$ devices, with j and i being the number of inputs and outputs, respectively. To compute the output i , O_i^1 , an activation function f is applied as $O_i^1 = f(\beta I_i^1)$, where β is a scaling factor that depends on the conductance range of the devices and the voltage range used to encode the input data. The outputs of the hidden layer feed the next layer of the crossbar array and a similar procedure produces the final outputs (currents) of the ANN. After the training of the network, the optimal values of conductance for each memristor in the array are known and it is possible to perform inference tasks.

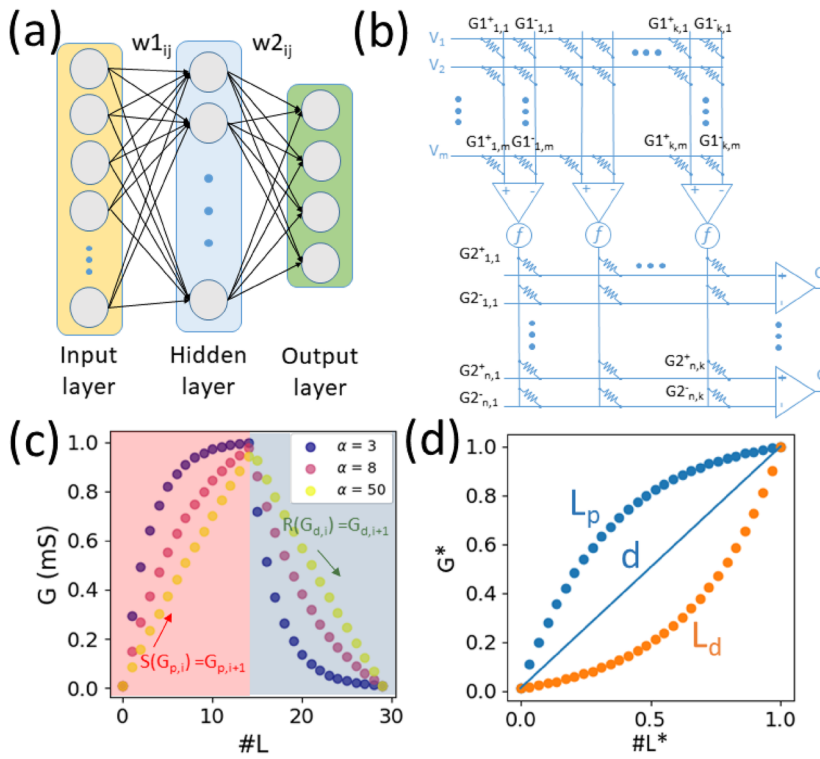


FIG. 1. (a) Schematic representation of an ANN with a single hidden layer. (b) Electrical scheme of a crossbar array architecture for the hardware implementation of the ANN shown in (a). (c) Synthetic potentiation (red region) and depression (blue region) curves obtained for different parameters α and fixed $\#L$. (d) Normalized P/D curves as a function of normalized L . To compute NLI, we consider d to be the length of the ideal (linear) segment connecting the first and last points of the P/D curves, while L_p and L_d represent the lengths of the potentiation and depression curves, respectively.

During the training procedure, the standard gradient descent (GD) method can be followed in order to compute the update of the synaptic weights Δw_{ij} . However, mapping these weight updates onto the conductance values of each differential pair is electrically challenging. A simpler alternative is the Manhattan rule,^{18,38} which is based on GD but enables conductance updates by applying fixed SET (increasing conductance) or RESET (decreasing conductance) voltage pulses at each training step. With this rule, we have a trade-off between learning precision and ease of implementation. In particular, the Manhattan rule dictates: (i) if $\Delta w_{ij} > 0$, a SET (RESET) voltage pulse is applied to modify G_{ij}^+ (G_{ij}^-); (ii) if $\Delta w_{ij} \leq 0$, a RESET (SET) voltage pulse is applied to modify G_{ij}^+ (G_{ij}^-). The Manhattan update rule allows mapping directly onto fixed-amplitude programming pulses in memristive hardware (where pulse polarity encodes the update direction), thereby avoiding the learning-rate tuning and pulse-modulation requirements typically associated with sign-SGD and enabling a more straightforward accounting of programming energy.

To update the conductance according to the Manhattan rule, we follow granular P/D curves, usually observed in memristors, synthetically generated according to $G_p = B(1 - \exp\{-n_i/\alpha\}) + G_{\min}$ for potentiation and $G_d = -B(1 - \exp\{(n_i - \#L)/\alpha\}) + G_{\max}$ for depression. In both cases, n_i is the discrete variable that represents the different conductance levels in the P/D curves, α is a parameter that controls its linearity, $G_{\min}$ is the lowest value of the conductance range of the device, and $\#L$ is the overall number of available levels in the conductance window. The parameter B is defined as $B = (G_{\max} - G_{\min})/(1 - \exp\{-\#L/\alpha\})$, where $G_{\max}$ is the

maximum conductance value. We will only consider symmetric synthetic curves, in the sense that both the potentiation and depression curves share the same number of levels $\#L$, conductance range, and parameter α .

In Fig. 1(c), we show synthetic P/D curves for $\#L = 15$ and $\alpha = 3, 8, 50$, where it can be seen that, for a fixed value of $\#L$, the parameter α controls the linearity of the curve, as larger values of α lead to more linear curves. However, if both parameters are varied simultaneously the linearity results from their combined effect—as expected from the exponential dependence on $\#L/\alpha$. To update the conductance values G_{ij}^+ and G_{ij}^- according to the Manhattan rule, we define two discrete functions, S and R , which reproduce the evolution of the P/D curves as SET or RESET voltage pulses are applied, respectively. To define these functions, we consider that the discrete values of the potentiation curve satisfy $G_{p,i} \leq G_{p,i+1}$, while those of the depression curve satisfy $G_{d,i} \geq G_{d,i+1}$. This is condensed in the ordered sequences $\{G_{p,i}\}_{i=1}^{\#L}$ and $\{G_{d,i}\}_{i=1}^{\#L}$. To capture this, we define $S(G_{p,i}) = G_{p,i+1}$ and $R(G_{d,i}) = G_{d,i+1}$. We also define $S(G_{p,\#L}) = G_{p,\#L}$ and $R(G_{d,\#L}) = G_{d,\#L}$ as a boundary condition, since in both cases, $G_{p,\#L}$ and $G_{d,\#L}$ are the last values of each sequence. Finally, since the learning rule might dictate a decrease(increase) in conductance even when the current value belongs to the potentiation (depression) curve, special care must be taken when applying updates to ensure consistency with the physical behavior of the device. For these cases, we define $S(G_{d,i}) = \min_j\{\|G_{d,i} - G_{p,j}\| | G_{d,i} < G_{p,j} < G_{p,\#L}\}$ and $R(G_{p,i}) = \min_j\{\|G_{p,i} - G_{d,j}\| | G_{d,1} < G_{d,j} < G_{p,i}\}$. We consider these functions to provide a good approximation of the general response of memristive devices to the accumulation of

voltage pulses, while preserving both the granularity and the distribution of the points that define the potentiation/depression (P/D) curves.

Different metrics can be proposed to quantify the degree of non-linearity of P–D curves, such as mean absolute value (MAE), root mean square error (RMSE) or maximum deviation;⁴¹ however, each one has its own limitations: MAE may underestimate the impact of localized nonlinearities by averaging errors over the entire curve; RMSE is strongly influenced by outliers or noise, which can overshadow the general shape of the trajectory; and maximum deviation captures only the largest pointwise discrepancy, making it highly sensitive to measurement artifacts and unrepresentative of the overall curve behavior.

We propose, instead, a non-linearity index (NLI) computed as follows: (i) we normalize $\#L$ as $\#L^*$ and G (G_d^* and G_p^*) so that they range from zero to one within each curve (this normalization is performed only for the NLI calculation); (ii) we compute the length d of the straight segment connecting the first and last points of the curve as $d = \sqrt{2}$; (iii) we calculate the lengths L_p and L_d along the potentiation and depression curves, respectively, as the sum of the distances between successive points: $L_{p/d} = \sum_{i=1}^{\#L^*-1} \sqrt{(n_{i+1}^* - n_i^*)^2 + (G_{(p/d)i+1}^* - G_{(p/d)i}^*)^2}$, where n_i^* is the normalized discrete variable that represents the different conductance levels in the P/D curves; and (iv) we compute $NLI = (L_{p/d} - d)/d$. This procedure is schematically illustrated in Fig. 1(d). Therefore, if $L_{p/d} \approx d$, the NLI approaches zero; otherwise, the NLI increases, as a signature of the greater non-linearity in the curve shape.

The main advantage of the NLI is that it offers a simple, model-independent, and unit-free metric to quantify how much the potentiation or depression trajectories deviate from an ideal linear trend, compactly capturing the overall curvature of the conductance evolution in a single index. A direct comparison between the proposed NLI and the conventional root-mean-square error (RMSE) metric (see [supplementary material](#) Fig. SM-1) reveals a clear monotonic

relationship, indicating that the NLI captures the same underlying nonlinearity trends in a metric-independent manner.

In Fig. 2(a), we show a heatmap of the NLI as a function of the parameters α and $\#L$ for potentiation curves, with both parameters varying in the range 0–300. We note that the number of levels in typical P/D curves is usually on the order of tens, although some reports have demonstrated up to several hundred levels,⁴² consistent with the range chosen for our simulations. Since we are considering symmetric P/D curves, the same heatmap applies to depression, and similar results are obtained for different conductance ranges (see Figs. SM-2). The white lines represent contour lines corresponding to constant NLI values, reflecting a linear relationship between $\#L$ and α . In Fig. 2(b), we show extracted P/D curves for points along the contour line corresponding to $NLI = 0.01$: $\#L = 50$, $\alpha = 48.83$ (yellow); $\#L = 100$, $\alpha = 98.55$ (light blue); and $\#L = 200$, $\alpha = 197.99$ (gray). The normalized curves retain the same shape along the contour, indicating that $\#L$ can be increased without altering the curve shape, provided that the parameters follow the linear relation defined by the contour. The same trend is observed in Fig. 2(c) for $NLI = 0.2$: $\#L = 50$, $\alpha = 5.85$ (green); $\#L = 100$, $\alpha = 11.82$ (blue); and $\#L = 200$, $\alpha = 23.76$ (red). Because the NLI is computed on normalized P/D curves, the heatmaps are independent of the absolute conductance range, reflecting linearity properties that depend only on the curve shape.

To estimate the energy dissipated by the crossbar during one epoch of training, we consider two contributions to the total energy E_t : one from the forward pass of the input images through the crossbar, E_f , and the other from the energy dissipated by each device during the conductance update, E_u , such that $E_t = E_f + E_u$. Assuming Joule heating as the primary dissipation mechanism, E_f can be computed as $E_f = \Delta t_f P_f$, where $\Delta t_f = 10$ ns, which is a typical target timescale for memristor writing and reading.^{43,44} The forward dissipated power is computed as $P_f = \sum_{\mu} \sum_i \sum_j (V_j^{\mu})^2 G_{ij}$. Here, $(V_j^{\mu})^2$ is the square of the j th component of the μ th input vector, and G_{ij} is the conductance matrix of the input layer. For a single perceptron (no hidden layers), the calculation is straightforward. However, for

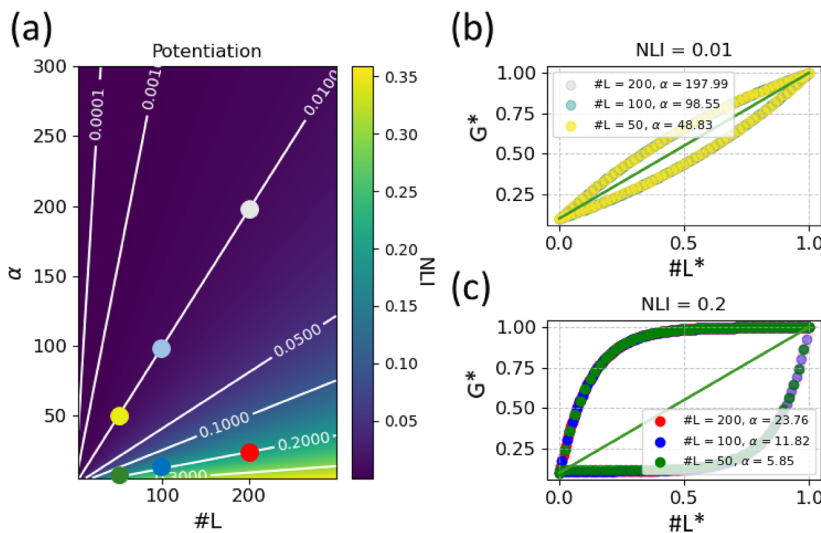


FIG. 2. (a) Heatmap corresponding to NLI as a function of parameters α and $\#L$. The white lines represent contour lines of the function. Only the heatmap for the potentiation is shown (depression presents the same heatmap). The extracted P/D curves for each combination of parameters (colored points) on the contour line for $NLI = 0.01$ and $NLI = 0.2$ are shown in panels (b) and (c), respectively, for normalized G^* and $\#L^*$. For points on the same contour line, the shape of the normalized P/D curve is similar to each other. Heatmaps are similar across different conductance ranges.

a MLP, only the energy dissipation in the input layer can be directly computed, since the output voltages of this layer are required to evaluate the next one.

Because we compute only the output currents of each layer, obtaining the corresponding output voltages would require solving a 2D resistor network for each input image at every epoch, which would make the simulations highly computationally expensive. To circumvent this problem, we estimate the energy dissipation in the output layer of the MLP by computing the average energy dissipated per memristor in the input layer and multiplying it by the number of memristors in the output layer.

On the other hand, the energy dissipated during the conductance update of a specific memristor is given by the integral $\int_{t_0}^{t_f} V_u^2(t) G_{ij}(t) dt$, where $V_u(t)$ is the applied voltage pulse (SET or RESET), $G_{ij}(t)$ is the time-dependent conductance of the (i, j) th memristor during the update, and t_0 and t_f define the initial and final times, respectively, of the time interval over which the update process takes place for each device. In order to compute the energy dissipated of the (i, j) th memristor during the update, E_{ij}^u , we use the trapezoidal rule to approximate the integral during the application of V_u using $E_{ij}^u \approx \Delta t_u / 2 (V_u^2(t_{k-1}) G_{ij}(t_{k-1}) + V_u^2(t_k) G_{ij}(t_k))$, where we choose $\Delta t_u = 10$ ns and $G_{ij}(t_k/t_{k-1})$ is the conductance of the (i, j) th memristor at the end/beginning of the update process. The total dissipated energy during one epoch is the sum over all devices $E_u = \sum_{ij} E_{ij}^u$.

III. RESULTS

To study the impact of the P/D curve parameters on the learning process performed in our simulated neural networks, we use a

standard machine learning benchmark: pattern recognition on the MNIST dataset, which consists of 60 000 images of 28×28 pixel digits from 0 to 9. We considered two network architectures: a simple perceptron (SP) with 28×28 input neurons and 10 output neurons, and a multilayer perceptron (MLP) with 28×28 input neurons, a hidden layer with 100 neurons, and an output layer with ten neurons. In both cases, a softmax activation function was used at the output layer, and a ReLU function was applied to the hidden layer of the MLP. Multiclass cross-entropy was used as the loss function.⁴⁵

To avoid overfitting during the learning process, we performed k-fold cross-validation¹ with $k = 5$, training on four folds and validating on the remaining one in each training realization. Each epoch consisted of processing all training samples divided into mini-batches of size 32.

The final accuracy after training vs NLI is shown in Fig. 3(a) for SP and in Fig. 3(b) for the MLP. Each curve in both figures corresponds to a different number of levels $\#L$ in the P/D curves. For fixed $\#L$, we see that the accuracy remains basically unchanged up to $\text{NLI} = 10^{-2}$ for SP and $\text{NLI} = 10^{-3}$ for MLP, decreasing afterward. This provides an estimate of how much nonlinearity in the P/D curves the crossbar array can tolerate before accuracy degrades. An improvement in accuracy is observed as $\#L$ increases. This makes sense since a higher density of levels brings the system closer to continuous updates, allowing for more precise convergence to the absolute minimum of the loss function. On the other hand, reducing the number of levels decreases the precision of updates, limiting the system's ability to find minima. From this, we can conclude that the number of available levels in the P/D curves plays a role similar to an effective learning rate.

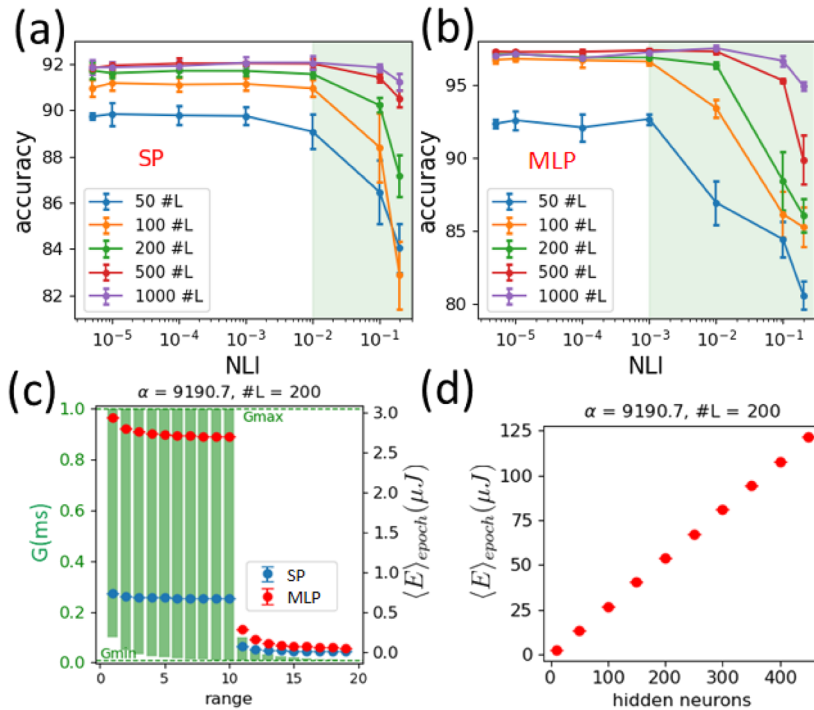


FIG. 3. Accuracy vs NLI for (a) SP and (b) MLP and $\#L$ spanning values of 50, 100, 200, 500, and 1000. The accuracy obtained remains almost constant for $\text{NLI} < 10^{-2}$ in the SP case and $\text{NLI} < 10^{-3}$ in the MLP case. For NLI values higher than these thresholds, the accuracy starts to decrease (green region). The same results were obtained for different ranges of conductance. (c) Average energy consumption per epoch (right scale) for different conductance ranges (left scale) for SP (blue dots) and MLP (red dots). The parameters of the P/D curves used to calculate the power consumption were $\alpha = 9190.7$ and $\#L = 200$. (d) Scaling of the average energy consumption per epoch as a function of the number of neurons in the hidden layer.

The highest accuracy was achieved with the MLP for NLI $\leq 10^{-3}$ and $\#L \geq 100$, reaching a value of $\approx 97\%$, which is very close to the benchmark value of $\approx 98\%$ reported for an ideal network with exact synaptic weight actualization.⁴⁶ The same trend was observed for different conductance ranges for SP (Fig. SM-3).

To assess the impact of realistic device behavior, we extended the analysis beyond the symmetric potentiation/depression (P/D) assumption used in the original model. In particular, we introduced controlled P/D asymmetry by keeping the potentiation branch strictly linear while progressively increasing the nonlinearity of the depression branch (see [supplementary material](#) Fig. SM-4). This approach captures a representative form of imbalance commonly observed in memristive devices while allowing a direct comparison with the symmetric baseline. The results indicate that learning accuracy is affected by P/D asymmetry, as expected, while the overall qualitative trends with NLI remain consistent with the symmetric case. In particular, the multilayer perceptron exhibits higher sensitivity to imbalance than the single perceptron, reinforcing the stricter linearity requirements identified for deeper networks. We note that asymmetry can arise through multiple mechanisms, including curvature mismatch, step-size imbalance, or differences in the number of accessible conductance states. A comprehensive exploration of these effects, together with device-to-device and cycle-to-cycle stochastic variability, is beyond the scope of the present work and will be addressed in future studies.

The estimated average energy consumption per epoch $\langle E \rangle_{\text{epoch}}$ is shown in Fig. 3(c) for both SP and MLP, using different conductance ranges between $G_{\text{max}} = 10$ mS and $G_{\text{min}} = 1$ mS, with parameters $\alpha = 9190.7$ and $\#L = 200$, in order to obtain linear P/D curves. From these results, we can see that for both SP and MLP, energy consumption is higher for conductance ranges where the highest values are closer to the maximum value G_{max} , which allows higher current values to be established in the crossbar. Conversely, as the conductance ranges move away from G_{max} and closer to G_{min} ,

the currents decrease, resulting in lower energy consumption, which is expected since the energy dissipation is linear on conductance. For all conductance ranges, the energy dissipated by the MLP is higher than that of the SP. This is expected, as the calculation involves summing the energy dissipation across all devices in the array. In all cases, we chose parameters assuring that accuracy takes its highest possible value (see Fig SM-5).

Finally, Fig. 3(d) demonstrates the average energy per epoch as a function of the number of neurons in the hidden layer. As anticipated, the energy increases linearly with the number of nodes, thereby revealing the direct influence of hidden-layer size on the overall energy consumption.

In Fig. 4(a) (first column), we plot the synaptic weight map (w_{0j}) together with the corresponding conductance maps (G_{0j}^+ and G_{0j}^-) for the SP, obtained after the training process and corresponding to the first output neuron. These weight maps, w_{mj} , indicate how much each input neuron j contributes to activating the output neuron m (in this case, $m = 0$). For SP architectures, each output neuron is associated with a specific digit class, so the weight map can be interpreted as a template or prototype representative of that class.

In the synaptic weight map w_{0j} , a well defined template of the digit 0 (associated with the first output neuron) can be observed. In addition, the synaptic weight values are highest at the periphery of the image and suddenly drop toward the center, reaching minimum values as the representative image of the class begins to take shape.

The same pattern can be found in the conductance map G_{0j}^+ , but interestingly, in the G_{0j}^- map, the pattern appears as a “negative image,” that is, the maximum values become minimum and vice versa. Since the conductances of both memristors in the differential pair are updated at each training step, it can be interpreted that the network stores the learned information redundantly, encoding the representative image of the class also in negative, through the G_{0j}^- conductance values of the differential pair. This suggests that the energy consumption during training can be reduced by eliminat-

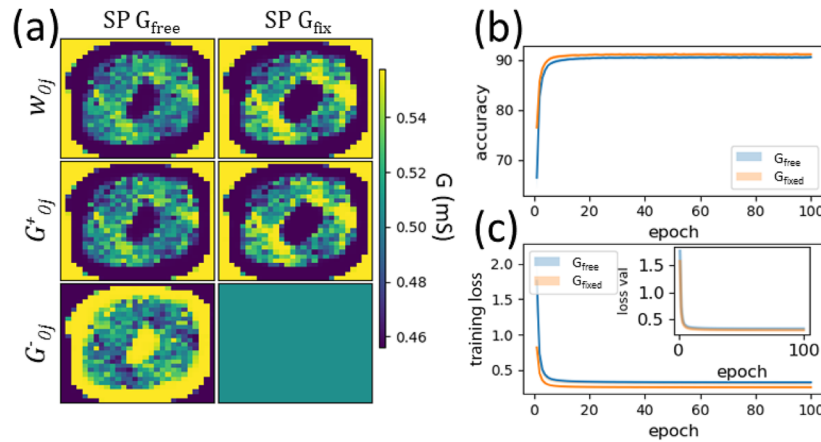


FIG. 4. [(a), first row] Synaptic weight map for the first output neuron of SP (w_{0j}), using both G_{free} (first column) and G_{fix} (second column) update methods. [(a), middle row] Conductance map for G_{0j}^+ memristors. Each column shows results for the same neurons and layers as in the top row. [(a), bottom row] Conductance map for the G_{0j}^- memristors; again, each column corresponds to the same neurons and layers as in the top row. (b) Accuracy evolution as a function of epochs. The solid line represents the mean values, and the shaded band indicates the standard deviations. The blue line corresponds to the G_{free} update method, while the orange line represents G_{fix} . (c) Training loss for both methods, with the validation loss shown in the inset. In both cases, convergence was achieved. A total of 200 training realizations were performed.

ing this redundant learning. One way to achieve this is by fixing the G_{ij}^- conductance to the midpoint of the conductance range defined by the P/D curves, and updating only the G_{ij}^+ values. We define this approach as G_{fix} , in contrast to G_{free} where all conductances are adjusted during training. The obtained maps from this learning process, using linear P/D curves, are similar to the previous one [Fig. 4 (second column)] except for G_{ij}^- , which remains constant in the fixed value. By examining the accuracy [Fig. 4(b)], training loss [Fig. 4(c)], and validation loss [Fig. 4(c), inset] curves as a function of epochs, we observe convergence in both cases (G_{free} in blue and G_{fix} in orange). Two hundred learning realizations were performed; the solid lines represent the mean curves, and the shaded bands indicate the standard deviation. Statistically, the small difference between the results (0.6%) suggests that both methods are equivalent in terms of learning performance on the crossbar.

For the hidden layers in a MLP, the interpretation of synaptic weight maps is not as straightforward as in the SP. In the case of the output layers, synaptic weight maps cannot be directly interpreted as class representatives due to the dimensional mismatch between the weight maps and the input images. Nevertheless, the strategy implemented in the SP case can still be applied to the synaptic weights of the MLP, by training G_{ij}^+ while fixing G_{ij}^- . This is illustrated in Fig. 5 (for the output layer, see Fig. SM-6). The evolution of accuracy, training loss, and validation loss under this approach is shown in Figs. 5(a)–5(c). We find that fixing G_{ij}^- results in only a minor decrease in accuracy, less than 0.58%.

Having established that fixing one synaptic weight in each differential pair does not significantly affect the learning performance of either SP or MLP, we next estimate the associated average energy savings. In the G_{fix} approach, only half of the memristors in the crossbar receive voltage pulses, directly reducing the energy required for training—the most time-consuming stage.

We notice that, for the parameter set considered here (pulse width, ranges, L), write events account for 90% and 95% of the total

energy in G_{fix} and G_{free} , respectively, as they adjust the synaptic weights and memristor conductivities. Since programming energy scales approximately with pulse width, using longer write pulses (50–100 ns) would further reinforce the dominance of write operations over reads in the considered training regime. Figure 6 summarizes the impact of the G_{fix} strategy on the overall average energy consumption during training, both for SP and MLP. The figure reports the percentage of energy reduction (ER) with respect to the conventional approach, in which both devices are updated at every training step, for a variety of memristor conductance ranges. The blue points correspond to SP, while red points correspond to MLP.

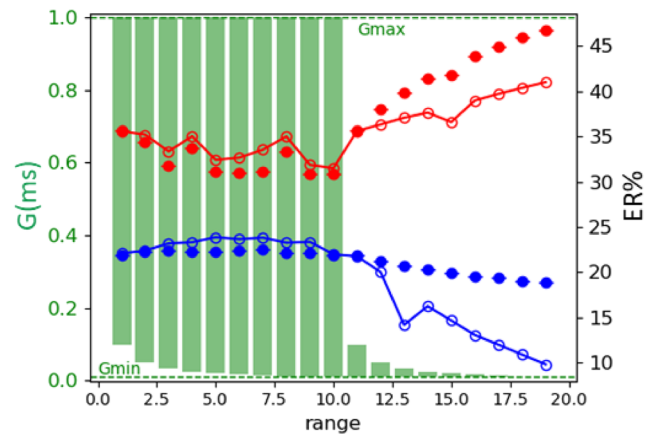


FIG. 6. Energy reduction (ER) for the G_{fix} method compared to G_{free} , for different conductance ranges. The solid blue dots represent the SP results, and the red dots represent the MLP. The open dots correspond to the estimated values of $\langle E^p \rangle_{epoch}$. The solid lines are shown for visualization purposes. One hundred training realizations were performed for each point.

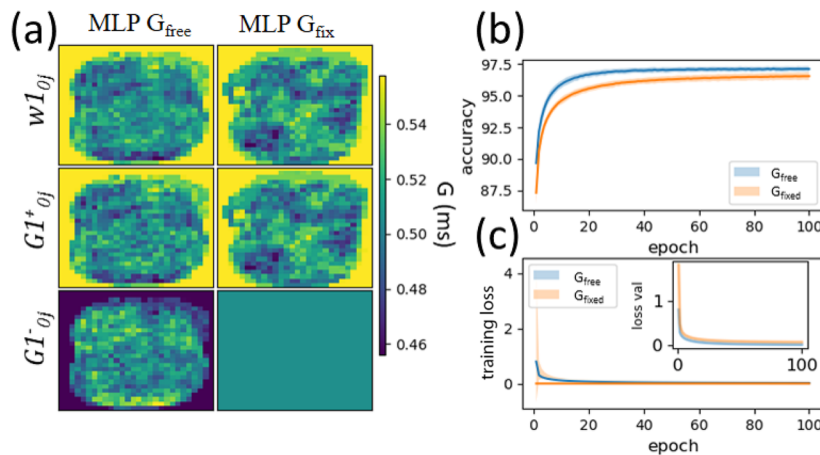


FIG. 5. [(a) first row] Synaptic weight ($w1_{0j}$) map for the first neuron of the hidden layer of MLP, using G_{free} update method (first column), and G_{fix} (second column). (Left, middle row) Conductance map for the $G1_{0j}^+$ memristors; each column shows results for the same neurons and layers as in the top row. (Left, bottom row) Conductance map for the $G1_{0j}^-$ memristors; again, each column corresponds to the same neurons and layers as in the top row. (b) Accuracy convergence as a function of epochs. The solid line represents the mean, and the shaded band indicates the standard deviation. The blue line corresponds to the G_{free} update method, while the orange line represents G_{fix} . (c) Training loss for both methods, with the validation loss shown in the inset. In both cases, convergence was achieved. A total of 200 training realizations were performed.

For the SP, the energy reduction, defined as $ER = (1 - E_{fix}/E_{free}) \times 100$, remains close to 20% across the first set of conductance ranges (where $G_{max} = 1$ mS and G_{min} varies between 0.1 mS and 10 μ S). When the conductance window is progressively narrowed (fixing G_{min} at 10 μ S and reducing G_{max} from 0.1 mS to 11.1 μ S), the ER slightly decreases to $\approx 15\%$ for the narrowest ranges. In contrast, the MLP consistently exhibits higher reductions, starting at 30%–35% for wide conductance ranges and increasing monotonically as the conductance window narrows and G_{max} drops below 0.1 mS. For the narrowest ranges considered, the energy savings exceed 45%.

As mentioned earlier, the reduction in energy consumption stems from fixing half of the memristors in each differential pair. However, the saving remains below 50% for both SP and MLP, reflecting differences in the conductance distributions between the free and fixed cases within the allowed conductance band. Indeed, Fig. SM-7 (for $G_{min} = 10$ μ S and $G_{max} = 100$ μ S) shows that a noticeable asymmetry develops in the fixed case, with a larger population of higher-conductance states—which are more energy demanding. This reduces the energy saving below the expectable 50% threshold.

To understand the distinct trends in energy reduction for SP and MLP as the conductance window narrows, we recall, first, that the dominant contribution to the energy per epoch arises from the application of writing pulses during training. As shown in the [supplementary material](#), this energy can be estimated as $\langle E^p \rangle_{epoch} \approx \Delta t V^2 \#D/2[\sum_k \langle G_{ij}(t_k) \rangle / \#E + \Delta G/2]$, where Δt is the time width of the writing pulse, V is the pulse amplitude, $\#D$ is the number of devices in the layer, $\#E$ is the number of training epochs, and ΔG is the conductance step in the P/D curves (assuming, for simplicity, the linear case). The energy is dominated by the term $\sum_k \langle G_{ij}(t_k) \rangle$, which represents the sum of the average device conductance over the k training epochs. Its evolution during training, which should ensure convergence and maximize accuracy, is directly linked to changes in the distribution of conductances G_{ij} and ultimately determines the overall training cost.

The observed ER trends arise from a subtle competition between the decay rates of E_{free} and E_{fix} , as shown in (Fig. SM-8). For SP (MLP), E_{fix} decreases with narrowing conductance windows at a higher (lower) rate than E_{free} , thereby reducing (increasing) ER. These differences ultimately reflect distinct evolutions of the G_{ij} distributions, as previously mentioned, in SP and MLP during training.

Overall, Fig. 6 highlights that while the G_{fix} strategy reduces training energy in both SP and MLP architectures, its benefits are particularly pronounced in deeper networks. This makes MLPs especially well suited to exploit this approach, achieving savings of up to 45% of the total training cost.

The energy savings reported for the proposed G_{fix} strategy are primarily driven by the reduction in write operations, which dominate the training energy in our framework (as mentioned before, $\sim 90\%$ – 95%). While solving the full resistive network would refine the absolute forward-pass energy, this contribution is non-dominant in the considered regime, and the relative advantage of G_{fix} —originating from the reduced number of programmed devices—is expected to remain largely preserved. For completeness, we mention that related energy-optimization studies in phase-change-memory-based spiking neural networks have been reported

in Refs. 47 and 48. In contrast, the present work focuses on MLP training with Manhattan updates, where the energy budget is dominated by discrete conductance programming events.

On the other hand, we note that fixing one device in each differential pair might introduce a possible trade-off between energy efficiency and long-term robustness. In the present simulations, the number of training epochs required for convergence remains essentially unchanged between the G_{free} and G_{fix} schemes [see Figs. 4(b), 4(c), 5(b), and 5(c)], indicating that the active device in G_{fix} does not experience an increased number of update cycles compared to the fully differential case. From an endurance perspective, G_{fix} concentrates conductance updates on a single device, which could, in principle, accelerate wear of that element relative to the complementary branch in G_{free} . Conversely, the fixed device may be more susceptible to long-term drift, as it is not periodically refreshed by programming pulses. These effects are expected to occur on timescales that extend beyond the training windows considered here and are, therefore, not captured by the present simulations. Accordingly, G_{fix} should be regarded as an energy-efficient operating mode rather than a universally optimal strategy. In scenarios requiring enhanced long-term stability—such as extended online learning, highly variable devices, or drift-sensitive applications—updating both devices or employing hybrid refresh strategies may be preferable.

Regarding network size, the present study focuses on a single-hidden-layer MLP with $\sim 10^2$ neurons to maintain tractable simulation times while enabling extensive parameter sweeps under realistic device models. As shown in Fig. 3(d), the training energy scales approximately linearly with the number of hidden neurons, indicating that the relative energy reduction enabled by G_{fix} is expected to remain valid for wider layers, even though the absolute energy increases. A systematic exploration of deeper architectures is left for future work.

To assess the robustness of the proposed framework beyond the MNIST benchmark, we extended the analysis to the more challenging Fashion-MNIST dataset while preserving the same input dimensionality and training protocol (see Fig. SM-9 of the [supplementary material](#)). Under these conditions, the single perceptron (SP) achieves a maximum accuracy of 82.4%, consistent with commonly reported ranges for this dataset, while the one-hidden-layer multilayer perceptron (MLP, 100 neurons) reaches 84.8%, within the typical performance window observed for hardware-constrained training schemes. Importantly, the qualitative trends identified for MNIST are preserved. In particular, the MLP remains more sensitive to update nonlinearity than the SP, requiring stricter NLI conditions for stable learning. At the same time, the proposed G_{fix} strategy introduces only minor accuracy degradation while maintaining its inherent energy-saving advantage, supporting the robustness of the main conclusions under more demanding classification conditions.

IV. DISCUSSION AND CONCLUSIONS

Our simulations provide a comprehensive assessment of the trade-offs involved in training memristor-based neural networks under realistic device constraints. By systematically varying the linearity of potentiation/depression curves, the conductance window, and the number of available conductance levels, we identified the device parameters that most strongly influence learning accuracy and energy efficiency. We found that both SP and MLP architectures

can tolerate a finite degree of nonlinearity in the P/D curves before accuracy begins to degrade, with MLPs requiring stricter linearity conditions ($NLI \leq 10^{-3}$) compared to SPs ($NLI \leq 10^{-2}$). Increasing the number of discrete conductance levels ($\#L$) effectively improves convergence, acting as a surrogate for a finer learning rate.

Nonetheless, our proposed G_{fix} strategy—where one memristor of each differential pair is fixed—demonstrates that energy consumption during training can be reduced by nearly 45% in MLPs without compromising accuracy, indicating that careful co-optimization of device operating windows and training schemes can yield substantial efficiency gains.

The energy savings enabled by G_{fix} primarily originate from the reduced number of programming events, which dominate the training energy in our framework (a few pJ per memristor update). The additional peripheral control required to hold one branch fixed (e.g., simple gating or multiplexing) is expected to remain well below the programming energy per synapse, consistent with reported energy scales in memristive crossbar systems and CMOS logic.^{49,50} Therefore, while peripheral circuitry will contribute to the absolute energy budget, it is not expected to significantly affect the relative energy advantage of G_{fix} in the write-dominated *in situ* training regime considered here.

Differential encoding provides not only information redundancy but also common-mode noise rejection and enhanced robustness to device fluctuations. Within this context, the proposed G_{fix} strategy can be interpreted as a controlled trade-off between noise immunity and energy efficiency: fixing one branch partially relaxes common-mode cancellation while substantially reducing programming energy. For the tasks and device parameters considered here, this trade-off yields significant energy savings with only a minor impact on accuracy.

From a broader efficiency perspective, the proposed approach is distinct from sparsity- and pruning-based methods. Network pruning typically targets parameter reduction and inference cost after training,⁵¹ while sparse-update schemes aim to reduce the number of weight modifications during training through data-dependent activity or gradient thresholds. In contrast, G_{fix} directly lowers the energy of *in situ* training by structurally reducing the number of programmed devices in differential pairs. As such, it is largely orthogonal and potentially complementary to pruning or sparsity techniques.

Overall, our study underscores the potential of the Manhattan update rule as a hardware-friendly training scheme that balances learning performance with energy efficiency. The results highlight that future memristive hardware should not only focus on improving device-level characteristics (linearity, resolution, and stability) but also exploit algorithmic strategies, such as redundant-weight fixing to minimize energy costs. These insights are particularly relevant for edge AI applications, where low-power and online learning capabilities are essential. We expect that combining realistic device engineering with training-rule co-design will pave the way toward scalable and energy-efficient neuromorphic hardware.

SUPPLEMENTARY MATERIAL

See the [supplementary material](#) for details on the approximations used to evaluate NN energy consumption, together with

additional numerical results corresponding to both SP and MLP performance.

ACKNOWLEDGMENTS

We acknowledge the support from Grant No. EU-H2020-RISE project MELON (Grant No. 872631). We gratefully thank Beatriz Noheda and the University of Groningen for the access to their high performance computation facilities.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

W. Quiñonez led the investigation and data curation and contributed to the formal analysis, conceptualization, and preparation of the original draft. M. J. Sánchez contributed to conceptualization, investigation, and funding acquisition and participated in the manuscript review. D. Rubi led the conceptualization and funding acquisition, contributed to the investigation, and co-led the preparation and review of the manuscript.

Walter Quiñonez: Conceptualization (equal); Data curation (lead); Formal analysis (equal); Investigation (lead); Writing – original draft (equal). **María José Sánchez:** Conceptualization (equal); Funding acquisition (equal); Investigation (equal); Writing – review & editing (equal). **Diego Rubi:** Conceptualization (lead); Funding acquisition (lead); Investigation (equal); Writing – original draft (lead).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

REFERENCES

- ¹A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. (O'Reilly Media, Inc., 2019).
- ²Y. LeCun, Y. Bengio, and G. Hinton, *Nature* **521**, 436 (2015).
- ³M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, *arXiv:1604.07316* (2016).
- ⁴G. Hinton, L. Deng, D. Yu, G. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. Sainath, and B. Kingsbury, *IEEE Signal Process. Mag.* **29**, 82 (2012).
- ⁵S. Zhang, L. Yao, A. Sun, and Y. Tay, *ACM Comput. Surv.* **52**, 5 (2019).
- ⁶A. Mehonic and A. J. Kenyon, *Nature* **604**, 255 (2022).
- ⁷X. Zou, S. Xu, X. Chen, L. Yan, and Y. Han, *Sci. China Inf. Sci.* **64**, 160404 (2021).
- ⁸S. Yu, *Neuro-Inspiring Computing Using Resistive Synaptic Devices* (Springer International Publishing, 2017).
- ⁹M. Zolfagharijad, U. Alegre-Ibarra, T. Chen, S. Kinge, and W. G. van der Wiel, *Eur. Phys. J. B* **97**, 70 (2024).
- ¹⁰A. Mehonic, D. Ielmini, K. Roy, O. Mutlu, S. Kvatinsky, T. Serrano-Gotarredona, B. Linares-Barranco, S. Spiga, S. Savel'ev, A. G. Balanov, N. Chawla, G. Desoli, G. Malavena, C. Monzio Compagnoni, Z. Wang, J. J. Yang, S. G. Sarwat, A. Sebastian, T. Mikolajick, S. Slesazek, B. Noheda, B. Dieny, T.-H. A. Hou, A. Varri, F. Brücknerhoff-Plückelmann, W. Pernice, X. Zhang, S. Pazos, M. Lanza,

- S. Wiefels, R. Dittmann, W. H. Ng, M. Buckwell, H. R. J. Cox, D. J. Mannion, A. J. Kenyon, Y. Lu, Y. Yang, D. Querlio, L. Hutin, E. Vianello, S. S. Chowdhury, P. Mannocci, Y. Cai, Z. Sun, G. Pedretti, J. P. Strachan, D. Strukov, M. Le Gallo, S. Ambrogio, I. Valov, and R. Waser, *APL Mater.* **12**, 109201 (2024).
- ¹¹ A. Sawa, *Mater. Today* **11**, 28 (2008).
- ¹² D. Ielmini and R. Waser, *Resistive Switching: From Fundamentals of Nanoionic Redox Processes to Memristive Device Applications* (Wiley-VCH, 2016).
- ¹³ W. Acevedo Román, C. Acha, M. Sánchez, P. Levy, and D. Rubi, *Appl. Phys. Lett.* **110**, 053501 (2017).
- ¹⁴ W. Román Acevedo, C. Ferreyra, M. J. Sánchez, C. Acha, R. Gay, and D. Rubi, *J. Phys. D: Appl. Phys.* **51**, 125304 (2018).
- ¹⁵ W. Román Acevedo, C. A. M. van den Bosch, M. H. Aguirre, C. Acha, A. Cavallo, C. Ferreyra, M. J. Sánchez, L. Patrone, A. Aguadero, and D. Rubi, *Appl. Phys. Lett.* **116**, 063502 (2020).
- ¹⁶ C. Ferreyra, M. Rengifo, M. J. Sánchez, A. Everhardt, B. Noheda, and D. Rubi, *Phys. Rev. Appl.* **14**, 044045 (2020).
- ¹⁷ F. Alibart, E. Zamanidoost, and D. B. Strukov, *Nat. Commun.* **4**, 2072 (2013).
- ¹⁸ M. Prezioso, F. Merrih-Bayat, B. Hoskins, G. Adam, K. Likharev, and D. Strukov, *Nature* **521**, 61 (2014).
- ¹⁹ P.-Y. Chen, B. Lin, I.-T. Wang, T.-H. Hou, J. Ye, S. Vrudhula, J.-s. Seo, Y. Cao, and S. Yu, in *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (IEEE, 2015), pp. 194–199.
- ²⁰ C. Li, D. Belkin, Y. Li, P. Yan, M. Hu, N. Ge, H. Jiang, E. Montgomery, P. Lin, Z. Wang, W. Song, J. P. Strachan, M. Barnell, Q. Wu, R. S. Williams, J. J. Yang, and Q. Xia, *Nat. Commun.* **9**, 2385 (2018).
- ²¹ V. Joshi, M. Le Gallo, S. Haeefi, I. Boybat, S. R. Nandakumar, C. Piveteau, M. Dazzi, B. Rajendran, A. Sebastian, and E. Eleftheriou, *Nat. Commun.* **11**, 2473 (2020).
- ²² S. Schmidgall, R. Ziaei, J. Achterberg, L. Kirsch, S. P. Hajisayedrazi, and J. Eshraghian, *APL Mach. Learn.* **2**, 021501 (2024).
- ²³ F. Aguirre, A. Sebastian, M. L. Gallo, W. Song, T. Wang, J. J. Yang, W. Lu, M.-F. Chang, D. Ielmini, Y. Yang, A. Mehonic, A. Kenyon, M. A. Villena, J. B. Roldán, Y. Wu, H.-H. Hsu, N. Raghavan, J. Suñé, E. Miranda, A. Eltawil, G. Setti, K. Smagulova, K. N. Salama, O. Krestinskaya, X. Yan, K.-W. Ang, S. Jain, S. Li, O. Alharbi, S. Pazos, and M. Lanza, *Nat. Commun.* **15**, 1974 (2024).
- ²⁴ P. Mannocci, M. Farronato, N. Lepri, L. Cattaneo, A. Glukhov, Z. Sun, and D. Ielmini, *APL Mach. Learn.* **1**, 010902 (2023).
- ²⁵ J. J. Yang, D. B. Strukov, and D. R. Stewart, *Nat. Nanotechnol.* **8**, 13 (2013).
- ²⁶ D. Ielmini, Z. Wang, and Y. Liu, *APL Mater.* **9**, 050702 (2021).
- ²⁷ G. W. Burr, R. M. Shelby, S. Sidler, C. di Nolfo, J. Jang, I. Boybat, R. S. Shenoy, P. Narayanan, K. Virwani, E. U. Giacometti, B. N. Kurdi, and H. Hwang, *IEEE Trans. Electron Devices* **62**, 3498 (2015).
- ²⁸ T. Shibata, T. Shinohara, T. Ashida, M. Ohta, K. Ito, S. Yamada, Y. Terasaki, and T. Sasaki, *Appl. Phys. Express* **13**, 043004 (2020).
- ²⁹ P. Apsangi, H. Barnaby, M. Kozicki, Y. Gonzalez-Velo, and J. Taggart, *Neuromorph. Comput. Eng.* **2**, 021002 (2022).
- ³⁰ D. Joksas, E. Wang, N. Barmapsalos, W. H. Ng, A. J. Kenyon, G. A. Constantinides, and A. Mehonic, *Adv. Sci.* **9**, 2105784 (2022).
- ³¹ W. Quiñonez, M. J. Sánchez, and D. Rubi, *Phys. Scr.* **98**, 095917 (2023).
- ³² T. Glint, G. Paul, A. Bende, R. Dittmann, and V. Rana, in *2024 31st IEEE International Conference on Electronics, Circuits and Systems (ICECS)* (IEEE, 2024), pp. 1–4.
- ³³ V. Yon, A. Amirsoleimani, F. Alibart, R. G. Melko, D. Drouin, and Y. Beilliard, *Front. Electron.* **3**, 825077 (2022).
- ³⁴ A. Gutsche, S. Siegel, J. Zhang, S. Hamsch, and R. Dittmann, *Front. Neurosci.* **15**, 661261 (2021).
- ³⁵ C. Bengel, F. Cüppers, M. Payvand, R. Dittmann, R. Waser, S. Hoffmann-Eifert, and S. Menzel, *Front. Neurosci.* **15**, 661856 (2021).
- ³⁶ Y. Kim, T. Gokmen, H. Miyazoe, P. Solomon, S. Kim, A. Ray, J. Doeven-speck, R. S. Khan, V. Narayanan, and T. Ando, *Front. Nanotechnol.* **4**, 1008266 (2022).
- ³⁷ H. Kim, M. R. Mahmoodi, H. Nili, and D. B. Strukov, *Nat. Commun.* **12**, 5198 (2021).
- ³⁸ E. Zamanidoost, F. M. Bayat, D. Strukov, and I. Kataeva, in *2015 IEEE 9th International Symposium on Intelligent Signal Processing (WISP) Proceedings* (IEEE, 2015), pp. 1–6.
- ³⁹ C. Li, M. Hu, Y. Li, H. Jiang, N. Ge, E. Montgomery, J. Zhang, W. Song, N. Dávila, C. E. Graves, Z. Li, J. P. Strachan, P. Lin, Z. Wang, M. Barnell, Q. Wu, R. S. Williams, J. J. Yang, and Q. Xia, *Nat. Electron.* **1**, 52 (2018).
- ⁴⁰ S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. (Prentice Hall, New York, 2009).
- ⁴¹ T. Iida, *Appl. Ocean Res.* **148**, 104024 (2024).
- ⁴² M. Xiao, M. Hellenbrand, N. Strkalj, B. Bakht, Z. Sun, N. Barmapsalos, D. Joksas, H. Dou, Z. Hu, P. Lu, S. Karki, S. Kunwar, J. D. Major, A. Chen, H. Wang, Q. Jia, A. Mehonic, and J. L. MacManus-Driscoll, *Adv. Funct. Mater.* **35**, 2418980 (2025).
- ⁴³ M. von Witzleben, S. Wiefels, A. Kindsmüller, P. Stasner, F. Berg, F. Cüppers, S. Hoffmann-Eifert, R. Waser, S. Menzel, and U. Böttger, *ACS Appl. Electron. Mater.* **3**, 5563 (2021).
- ⁴⁴ H. Jiang, L. Han, P. Lin, Z. Wang, M. H. Jang, Q. Wu, M. Barnell, J. J. Yang, H. L. Xin, and Q. Xia, *Sci. Rep.* **6**, 28525 (2016).
- ⁴⁵ I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016).
- ⁴⁶ B. Illing, W. Gerstner, and J. Brea, *Neural Networks* **118**, 90 (2019).
- ⁴⁷ S. B. Hamid, A. I. Khan, H. Zhang, A. V. Davydov, and E. Pop, *IEEE Electron Device Lett.* **45**, 1819 (2024).
- ⁴⁸ S. Ambrogio, N. Ciochini, M. Laudato, V. Milo, A. Pirovano, P. Fantini, and D. Ielmini, *Front. Neurosci.* **10**, 56 (2016).
- ⁴⁹ S. Ambrogio, P. Narayanan, H. Tsai, R. M. Shelby, I. Boybat, C. di Nolfo, S. Sidler, M. Giordano, M. Bodini, N. C. P. Farinha, B. Killeen, C. Cheng, Y. Jaoudi, and G. W. Burr, *Nature* **558**, 60 (2018).
- ⁵⁰ M. Horowitz, in *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)* (IEEE, 2014), pp. 10–14.
- ⁵¹ D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Guttag, in *Proceedings of Machine Learning and Systems 2 (MLSys, 2020)*, https://proceedings.mlsys.org/paper_files/paper/2020/hash/6c44dc73014d66ba49b28d483a8f8b0d-Abstract.html.